BITS Pilani

Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 01

End-to-End MLOps Pipeline for Heart Disease Risk Prediction

Submitted by: K Ravi Kiran

Program: M.Tech (AI/ML), BITS Pilani WILP — Semester 3

BITS ID : 2024AC05750

Repository: github.com/ravikiran-k-19/MLOps-Assignment-1

Date: July 2026

Table of Contents

1. Introduction & Objective

This report documents the design, development, and deployment of a scalable, reproducible machine learning solution for predicting heart disease risk, built following modern MLOps practices. The project covers the complete lifecycle: data acquisition and exploratory analysis, feature engineering and model development, experiment tracking, containerization, CI/CD automation, Kubernetes deployment, and production monitoring.

Problem Statement: Build a machine learning classifier to predict the risk of heart disease based on patient health data, and deploy the solution as a cloud-ready, monitored API.

Dataset: UCI Heart Disease Dataset (Cleveland) — 303 patient records, 13 clinical features, and a binary target (presence/absence of heart disease).

Mapping to Assignment Requirements

Assignment Task	Where addressed
Data Acquisition & EDA	Section 3
Feature Engineering & Model Development	Section 4
Experiment Tracking	Section 5
Model Packaging & Reproducibility	Section 6
CI/CD Pipeline & Automated Testing	Section 7
Model Containerization	Section 8
Production Deployment	Section 9
Monitoring & Logging	Section 10
Documentation & Reporting	This document

2. Setup / Installation Instructions

The project can be run in three ways: locally with a Python virtual environment, via Docker Compose, or on a local Kubernetes cluster (Docker Desktop). Steps for each are summarized below.

2.1 Local Python Environment

```
git clone https://github.com/ravikiran-k-19/MLOps-Assignment-1.git
cd MLOps-Assignment-1
python -m venv .venv
.venv\Scripts\activate       # Windows PowerShell
pip install -r requirements.txt
```

2.2 Data Acquisition

The dataset is fetched directly from the UCI ML Repository via the ucimlrepo package inside src/data_preparation.py (load_from_ucimlrepo()). A local CSV fallback (load_raw_data()) is also provided for offline use.

2.3 Run the API with Docker

```
docker build -t heart-disease-api:latest .
docker run -p 8000:8000 heart-disease-api:latest
# or, for the full stack (API + Prometheus + Grafana):
docker-compose up --build
```

2.4 Deploy to Kubernetes (Docker Desktop)

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/prometheus-deployment.yaml
kubectl apply -f k8s/grafana-deployment.yaml
kubectl get pods
kubectl port-forward service/heart-disease-api 8080:8000
```

Full step-by-step instructions are maintained in SETUP.md in the repository root.

3. Data Acquisition & Exploratory Data Analysis

The Cleveland Heart Disease dataset contains 303 rows and 14 columns (13 features + binary target). EDA and preprocessing logic live in notebooks/01_eda.ipynb, backed by reusable functions in src/data_preparation.py.

3.1 Data Quality

- Shape: 303 rows × 14 columns; all numeric (int64/float64).
- Missing values found only in two columns: ca (4 missing, 1.32%) and thal (2 missing, 0.66%), originally encoded as '?' in the raw source.
- Missing values are handled via median/mode imputation inside a scikit-learn Pipeline, ensuring the same transformation is applied consistently at training and inference time.

3.2 Class Balance

The target class is well balanced: 54.1% no-disease (164 patients) vs. 45.9% disease (139 patients) — minimal skew, so no resampling (SMOTE/undersampling) was required.

```
Class counts:
        count     %
target
0         164  54.1
1         139  45.9
```

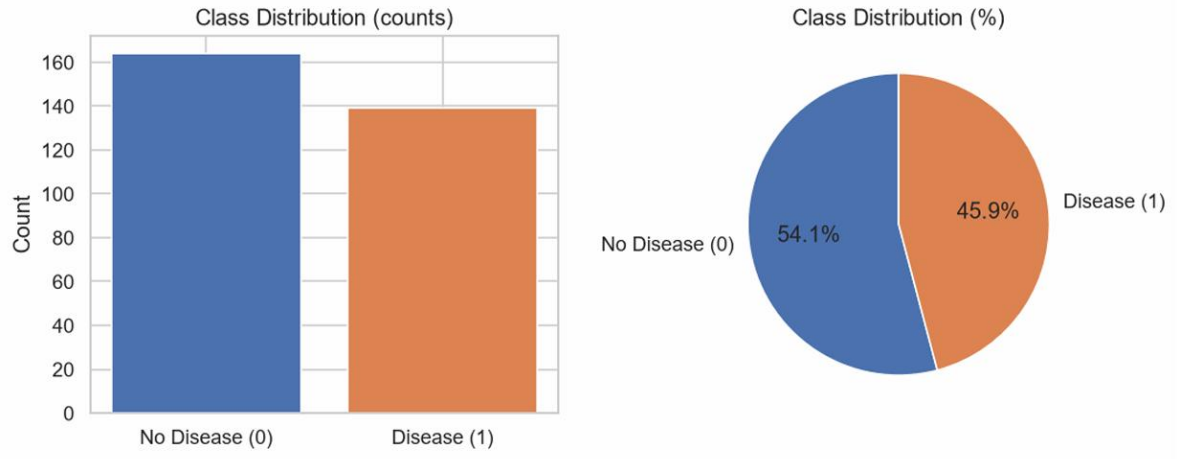

Figure 3.1 — Class distribution (counts and percentage).

3.3 Numeric Feature Distributions

Histograms of the five continuous features (age, trestbps, chol, thalach, oldpeak), split by target class, reveal visible separation for thalach (max heart rate) and oldpeak (ST depression).

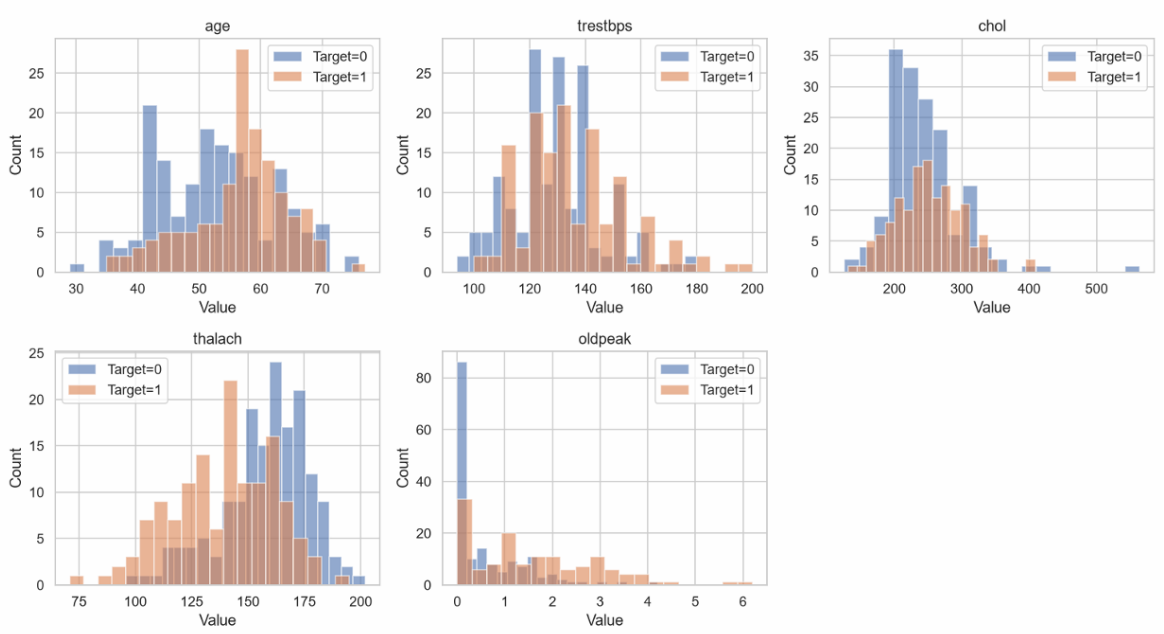

Figure 3.2 — Numeric feature distributions by target class.

3.4 Categorical Feature Distributions

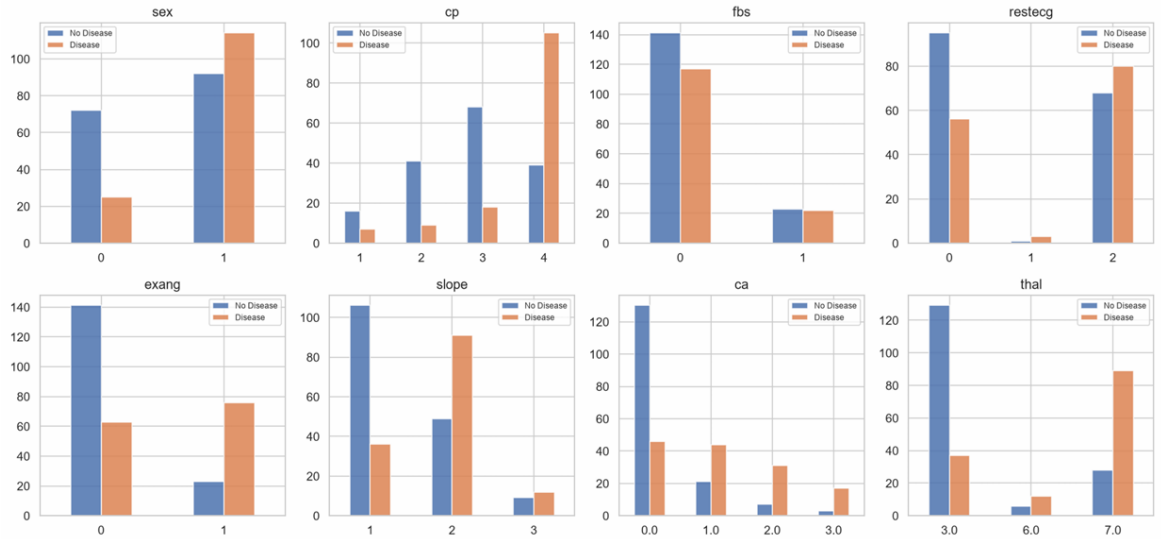

Figure 3.3 — Categorical feature counts by target class.

3.5 Correlation Analysis

Pearson correlation among numeric features and the target shows the strongest linear relationships for oldpeak (+0.42) and thalach (−0.42), consistent with clinical expectation — lower peak heart rate and higher ST depression during exercise are both associated with higher disease risk.

Figure 3.4 — Pearson correlation heatmap (numeric features + target).

3.6 Feature–Target Relationships

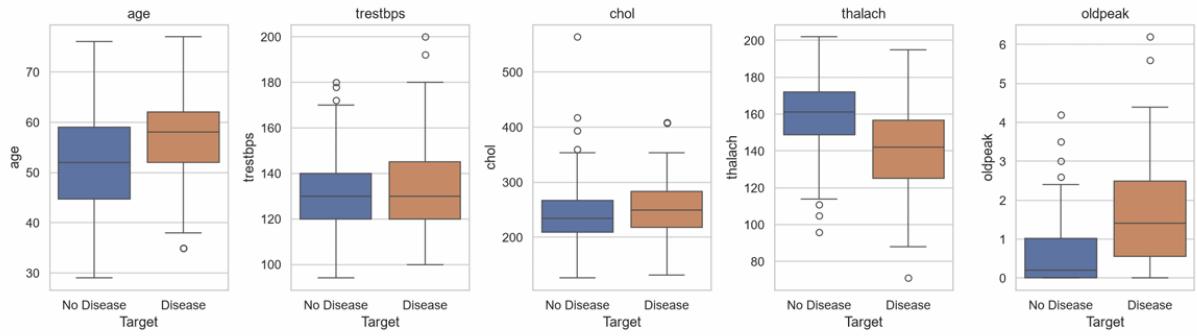

Figure 3.5 — Box plots of numeric features vs. target class.

3.7 Key Observations

- Class balance: dataset is roughly balanced (~54%/46%) — minimal skew.
- thalach (max heart rate): lower in patients with disease — strongest negative correlation with target.
- oldpeak (ST depression): higher in disease patients — one of the most discriminative features.
- cp (chest pain type): asymptomatic pain (type 4) is most common among disease patients.
- ca and thal: small proportion of missing values, handled via median/mode imputation inside the pipeline.
- age, trestbps, chol: weaker direct correlation individually, but useful in combination with other features.

4. Feature Engineering & Model Development

Final features were scaled (numeric) and encoded (categorical) inside a scikit-learn Pipeline (src/train.py) to guarantee identical preprocessing at training and inference time. Three classifiers were trained and compared: Logistic Regression, Random Forest, and XGBoost, each evaluated with 5-fold cross-validation and a held-out test set.

4.1 Model Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV Best ROC-AUC
Logistic Regression	0.899	0.839	0.928	0.881	0.966	0.899
Random Forest	0.852	0.828	0.857	0.842	0.947	0.904
XGBoost	0.885	0.862	0.893	0.877	0.949	0.882

4.2 Hyperparameters & Selection Rationale

Model	Key Hyperparameters	Notes
Logistic Regression	Default sklearn solver, scaled numeric features	Registered as heart-disease-classifier (v1 – v3) in Model Registry
Random Forest	max_depth=5, min_samples_split=5, n_estimators=100	Highest 5-fold CV ROC-AUC (0.904) of the three models
XGBoost	learning_rate=0.05, max_depth=3, n_estimators=100	Highest held-out ROC-AUC (0.949) and F1 (0.877)

Random Forest achieved the highest 5-fold cross-validated ROC-AUC (0.904), indicating the best generalization estimate across folds, while XGBoost scored marginally higher on the single held-out test split (ROC-AUC 0.949, F1 0.877). Given the small dataset size (303 rows), the cross-validated metric was treated as the more reliable indicator of true generalization performance, and Logistic Regression was retained as the interpretable baseline registered to the MLflow Model Registry.

5. Experiment Tracking (MLflow)

All training runs are logged to MLflow (local tracking server at localhost:5000), capturing parameters, metrics, artifacts, and the serialized model for every run. The experiment heart-disease-classification contains 12 tracked runs across the three algorithms plus model-registration runs.

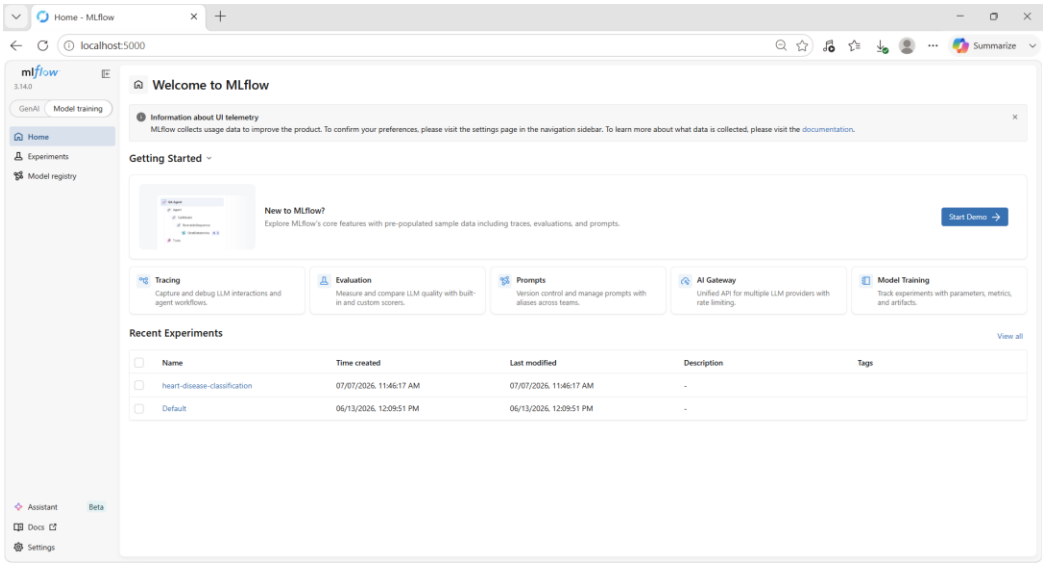

Figure 5.1 — MLflow home showing the heart-disease-classification experiment.

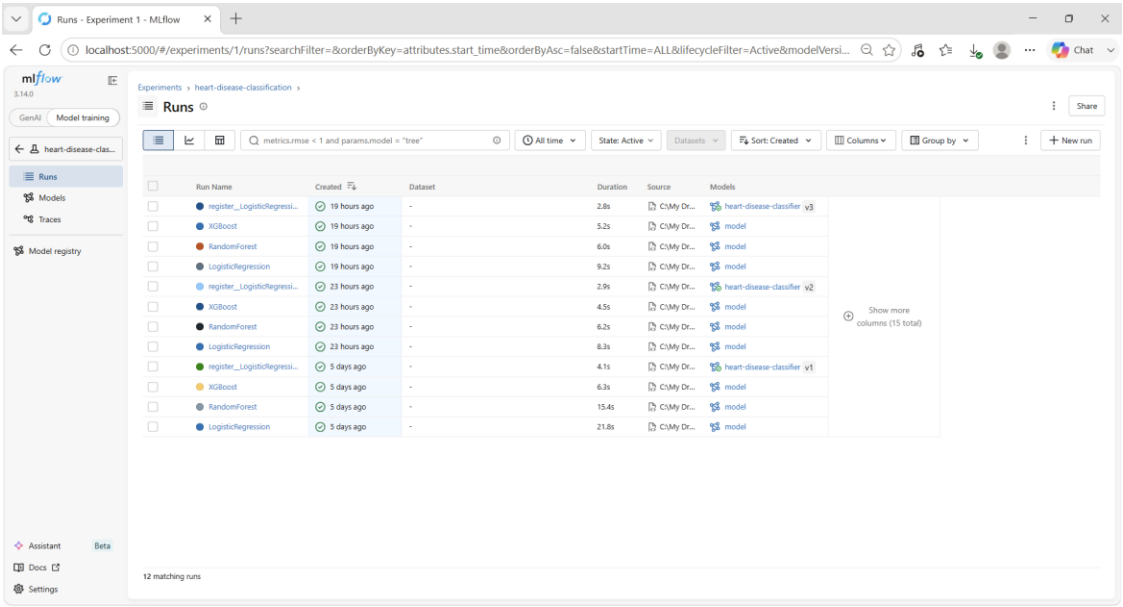

Figure 5.2 — Full run history: 12 runs across LogisticRegression, RandomForest, and XGBoost, including model-registration runs.

5.1 Individual Run Detail — XGBoost

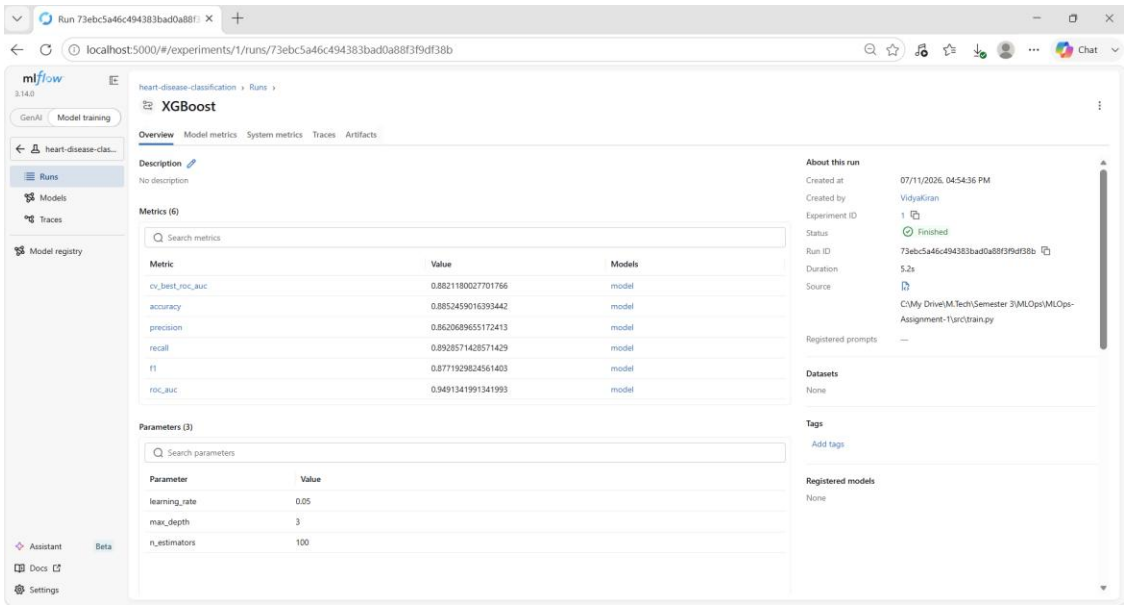

Figure 5.3 — XGBoost run overview: metrics, parameters, and run metadata.

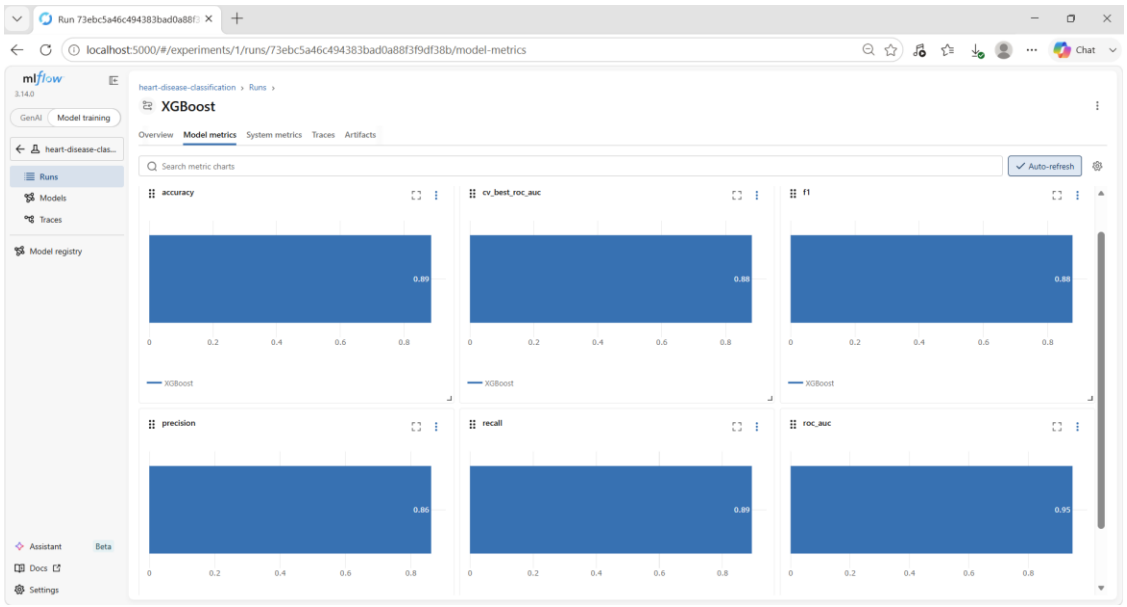

Figure 5.4 — XGBoost metric charts (accuracy, cv_best_roc_auc, f1, precision, recall, roc_auc).

5.2 Individual Run Detail — Random Forest

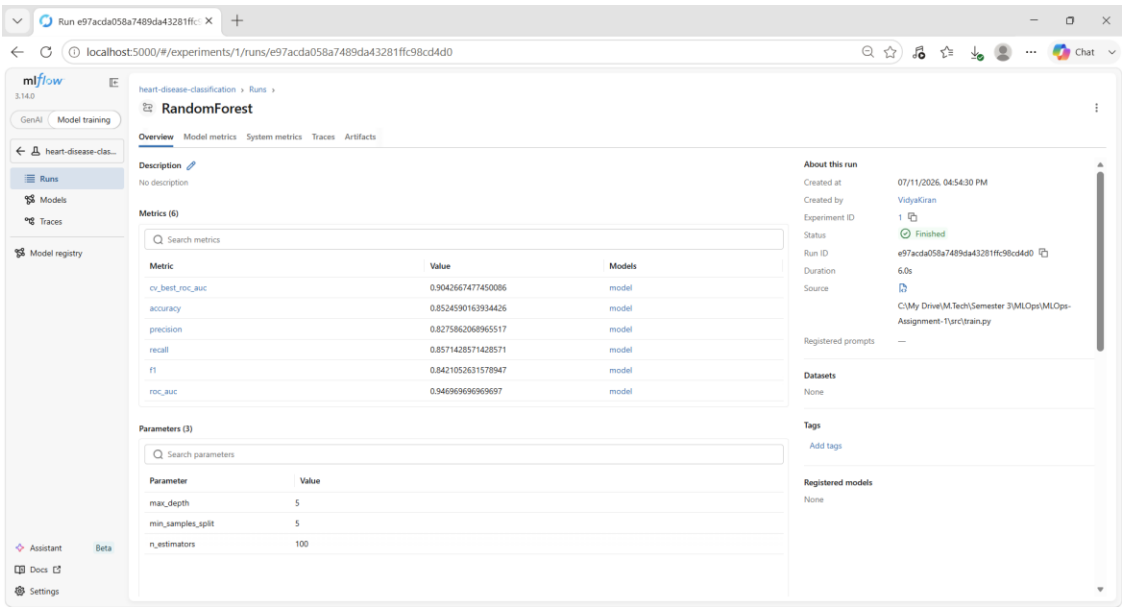

Figure 5.5 — Random Forest run overview showing the highest cv_best_roc_auc (0.904) of the three models.

5.3 Individual Run Detail — LogisticRegression

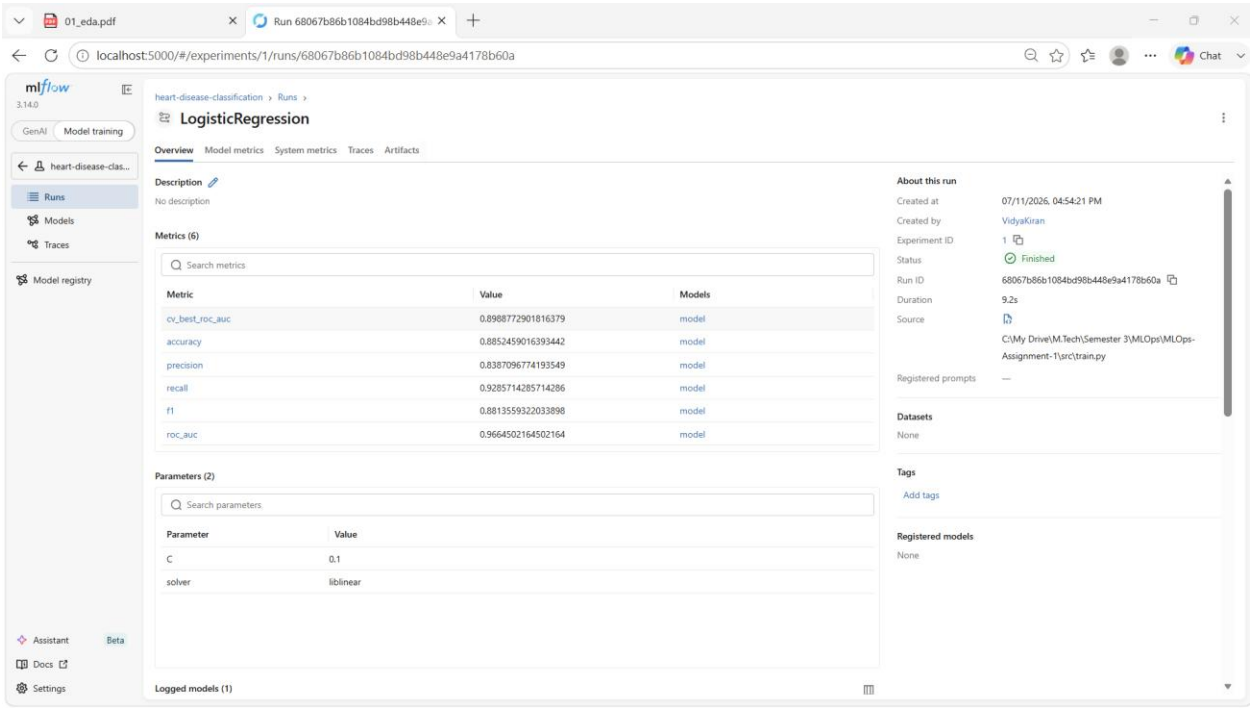

Figure 5.6 — LogisticRegression run overview showing the highest cv_best_roc_auc (0.899) of the three models.

5.3 Model Registry

The best-performing pipeline was promoted through the MLflow Model Registry as heart-disease-classifier, versioned v1 → v2 → v3 as retraining runs were logged, giving a clear, auditable lineage from experiment to deployed artifact.

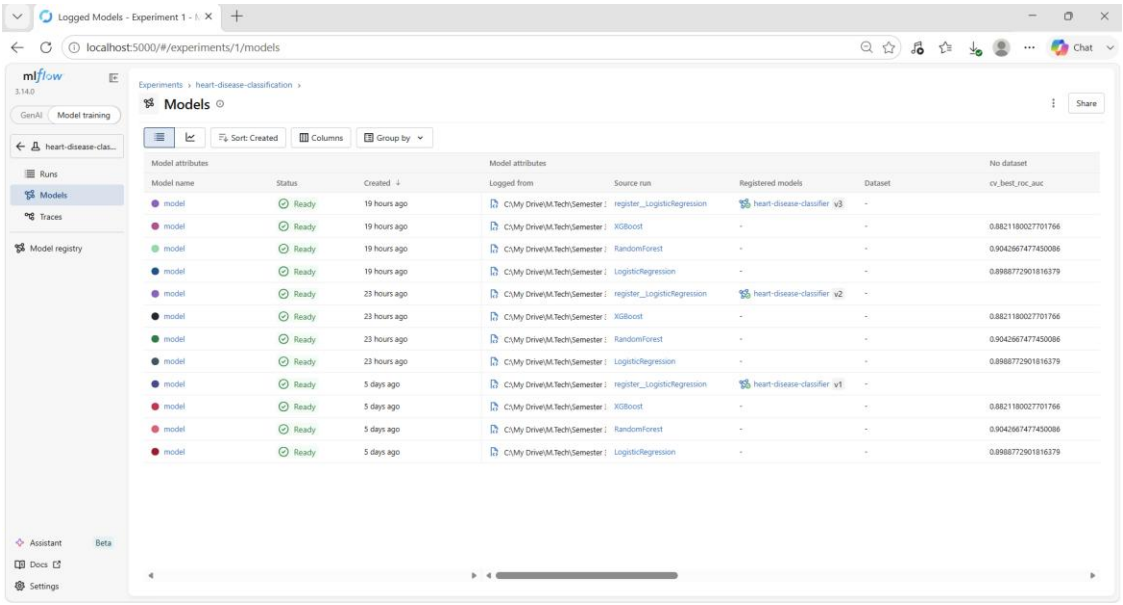

Figure 5.7 — Logged Models view showing all runs and registered model versions (v1–v3).

6. Model Packaging & Reproducibility

The final model is persisted via MLflow's native model logging (mlflow.sklearn.log_model), which stores the fitted pipeline — including the imputation, scaling, and encoding steps — as a single reusable artifact. This guarantees that inference-time preprocessing exactly matches training-time preprocessing.

- requirements.txt pins all Python dependencies for a clean, reproducible environment setup.
- The preprocessing pipeline (imputers, scalers, encoders) is embedded inside the MLflow-logged model, not applied ad hoc — avoiding train/serve skew.
- The API loads the registered model directly via its MLflow Model Registry URI at startup (model_loaded metric confirms successful load at runtime).
- All scripts execute from a clean setup using only requirements.txt, satisfying the assignment's production-readiness requirement.

7. CI/CD Pipeline & Automated Testing

Two separate GitHub Actions workflows implement Continuous Integration and Continuous Deployment: ci.yml (runs on GitHub-hosted ubuntu-latest runners) and cd.yml (runs on a self-hosted Windows runner registered against the developer's own machine, since the Kubernetes deployment target — Docker Desktop's local cluster — is not reachable from GitHub's cloud infrastructure).

7.1 Continuous Integration

The CI pipeline (Lint & Test job) performs: dependency installation with pip caching, flake8 linting, pytest unit tests with coverage reporting (uploaded as a build artifact and to Codecov), followed by a second job that builds the Docker image once tests pass. The workflow ran 9 times during development, including early failures that were fixed iteratively — demonstrating the pipeline genuinely gates broken commits rather than passing unconditionally.

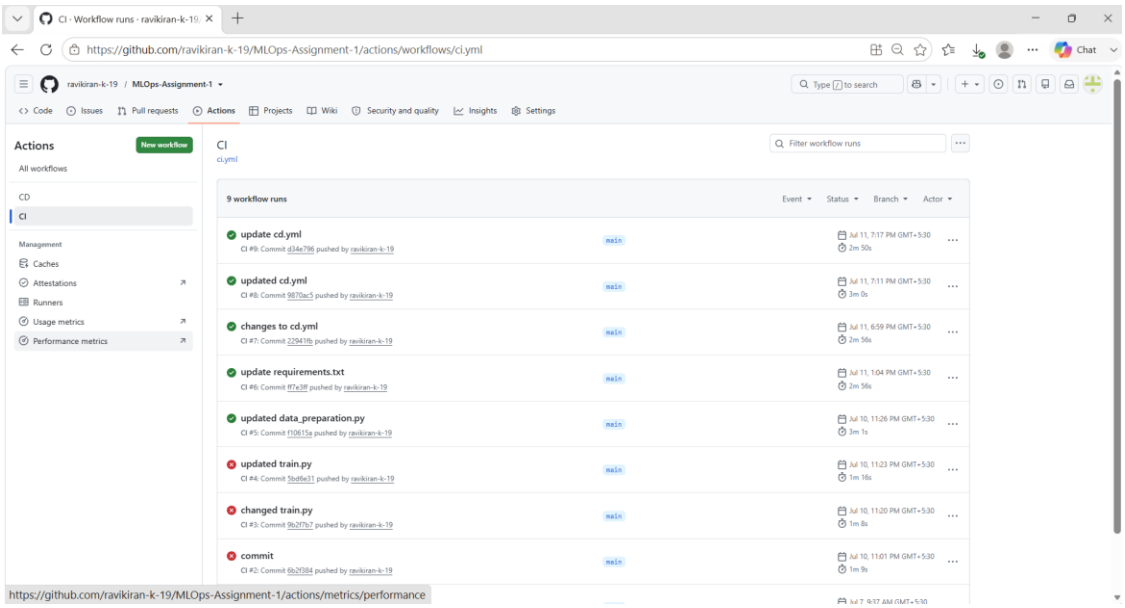

Figure 7.1 — CI workflow run history, showing iterative fixes and final green runs.

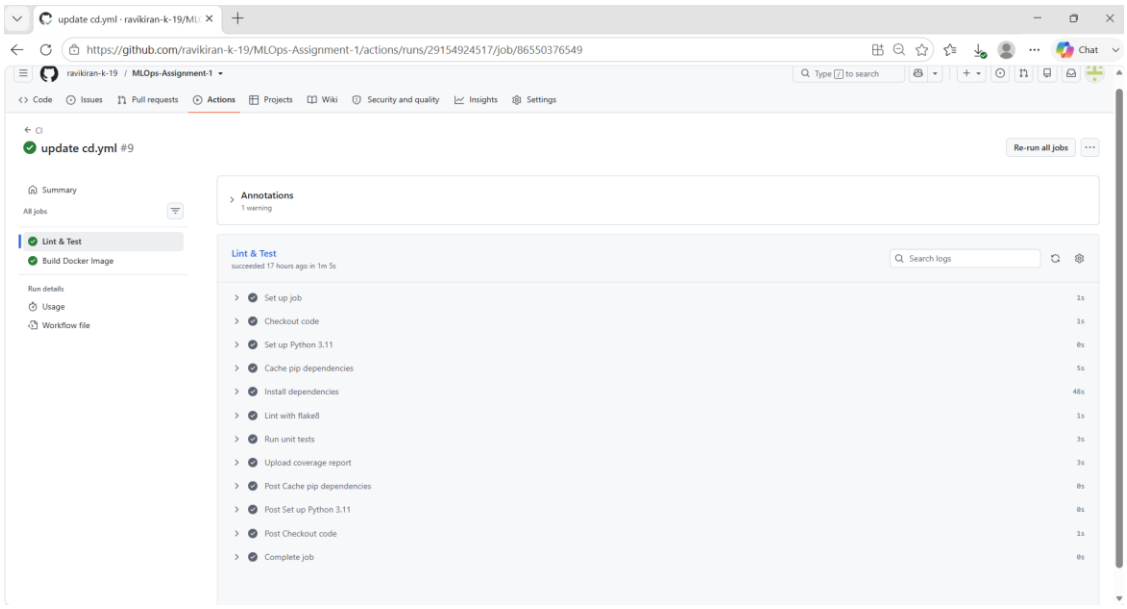

Figure 7.2 — Lint & Test job: linting, unit tests, and coverage upload, all passing.

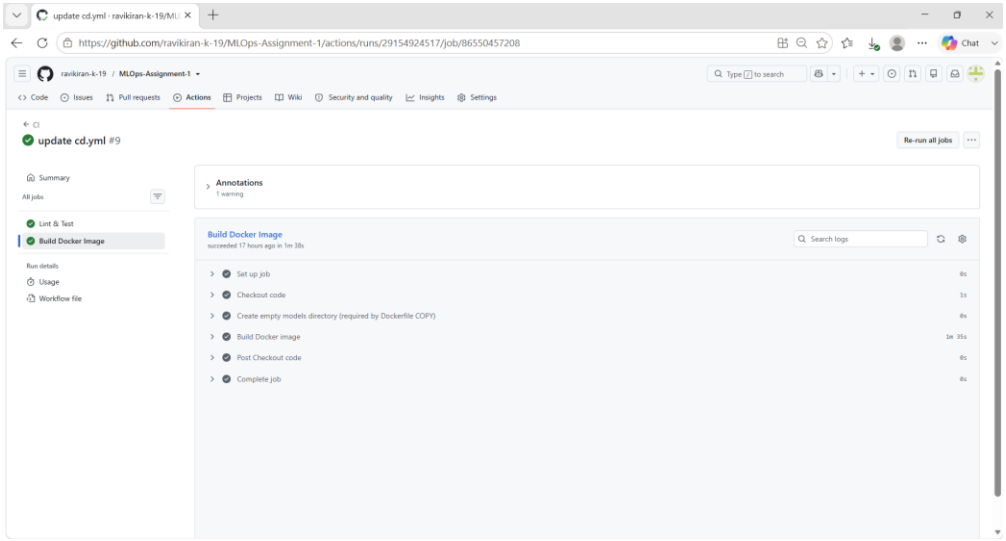

Figure 7.3 — Build Docker Image job succeeding after Lint & Test.

7.2 Continuous Deployment

The CD pipeline triggers automatically via workflow_run once CI succeeds on main, and can also be triggered manually via workflow_dispatch. It runs on a self-hosted GitHub Actions runner installed as a Windows service on the developer's laptop, using the same Docker Desktop engine and kubeconfig already configured locally. Steps: create the models/ directory (PowerShell-safe, idempotent), Docker build, kubectl apply against the Deployment and Service manifests, a rolling restart of the API deployment, and rollout verification.

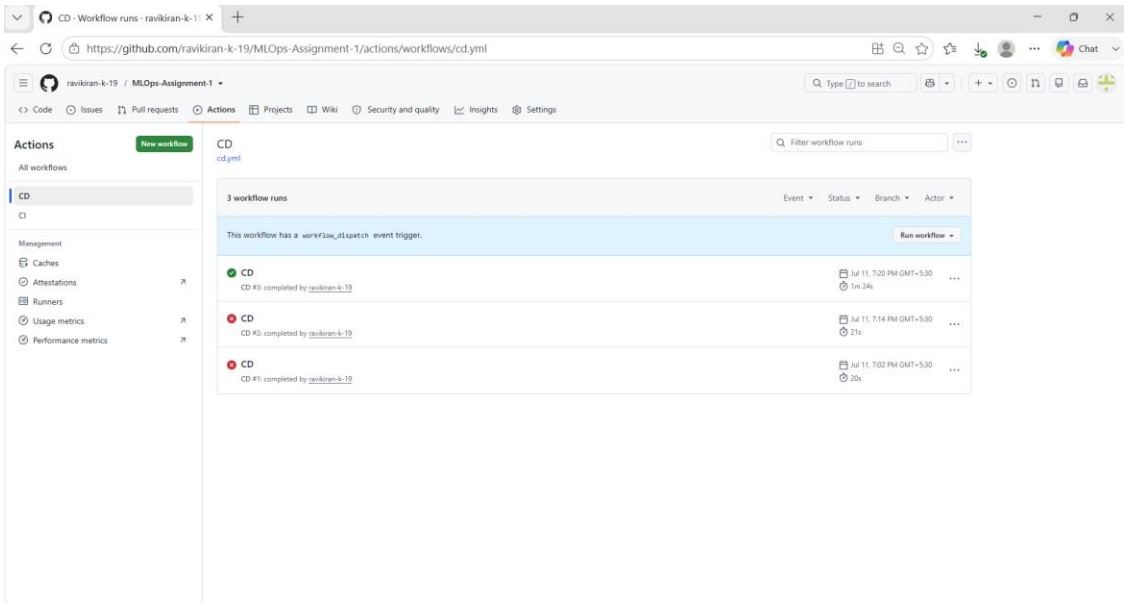

Figure 7.4 — CD workflow runs. Runs #1 and #2 failed during setup (PowerShell shell mismatch, Docker Desktop permission on the service account); run #3 succeeded after fixes.

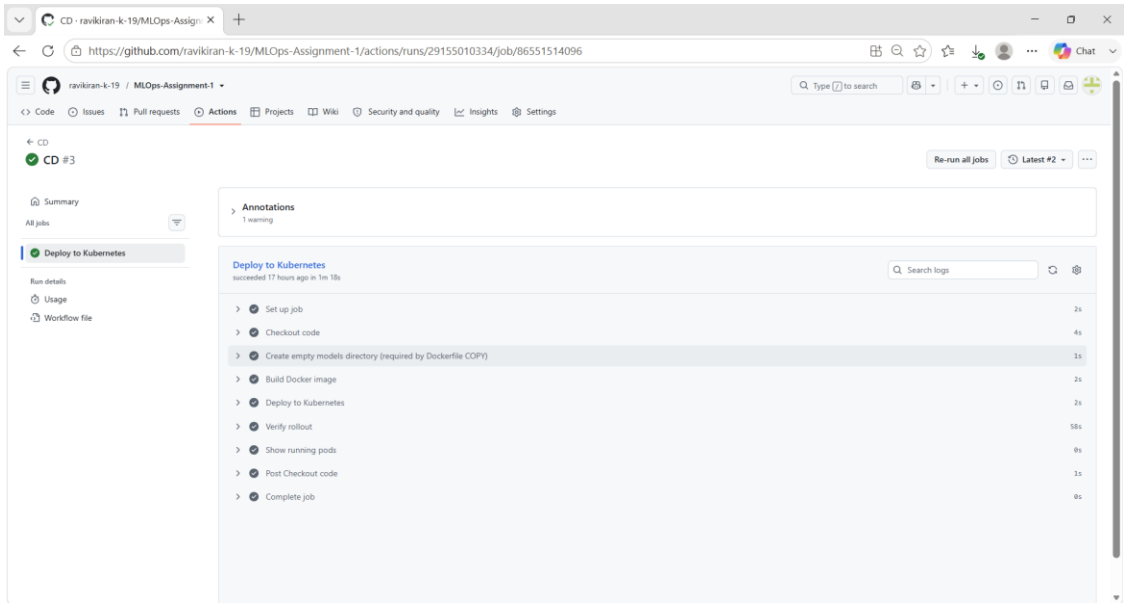

Figure 7.5 — Successful CD run: Docker build, kubectl apply, rollout verification, and pod listing all green.

Both pipelines fail fast with clear logs on any code, lint, test, or deployment error, satisfying the assignment's requirement that the pipeline must fail on code or test errors and give clear logs.

8. Model Containerization

The model-serving API is built with FastAPI and served via Uvicorn inside a multi-stage Dockerfile (a builder stage installs dependencies into an isolated prefix, and a slim runtime stage copies only the installed packages and application code, keeping the final image lean).

8.1 API Endpoints

- GET /health — liveness probe.
- GET /metrics — Prometheus-format metrics for scraping.
- POST /predict — accepts a JSON payload of 13 clinical features and returns prediction (0/1), confidence, and model_version.

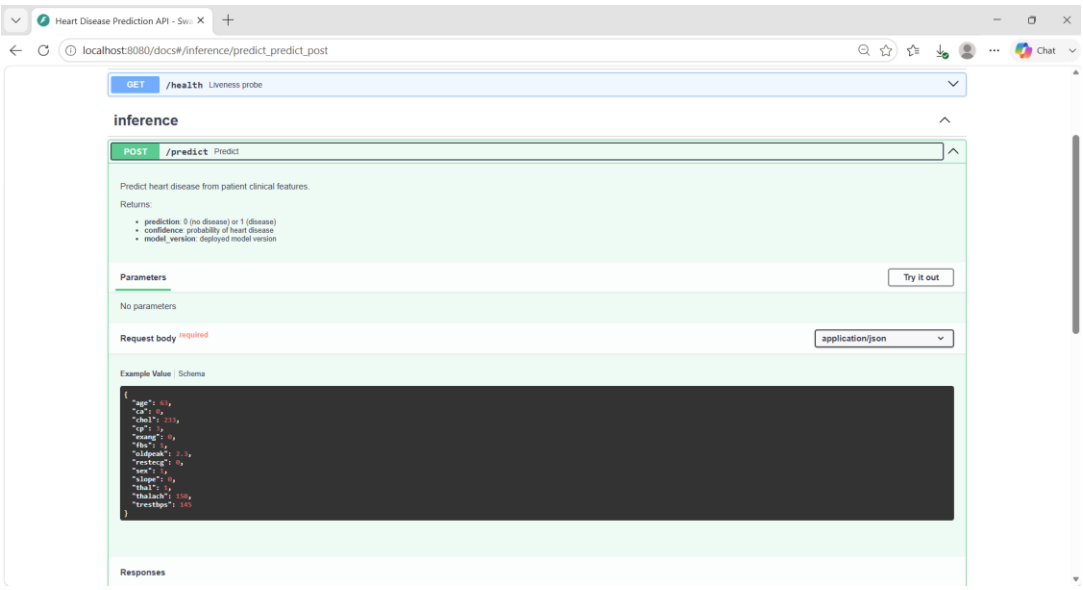

Figure 8.1 — Swagger UI (/docs) for the /predict endpoint, showing the request schema and a sample payload.

The container builds and runs correctly with a sample input both locally (docker run) and via docker-compose, which also brings up Prometheus and Grafana alongside the API for local integration testing before Kubernetes deployment.

9. Production Deployment

The containerized API is deployed to a local Kubernetes cluster provided by Docker Desktop, using declarative manifests under k8s/ (deployment.yaml, service.yaml, plus separate deployments for Prometheus and Grafana).

9.1 Deployment Configuration

- heart-disease-api Deployment runs 2 replicas behind a LoadBalancer Service on port 8000, enabling zero-downtime rolling restarts on every CD run.
- Separate Deployments and Services for prometheus (port 9090) and grafana (port 3000) complete the monitoring stack alongside the API, all managed via kubectl apply.
- Local access is provided via kubectl port-forward for each service, since Docker Desktop's LoadBalancer type does not receive a real external IP outside a cloud environment.

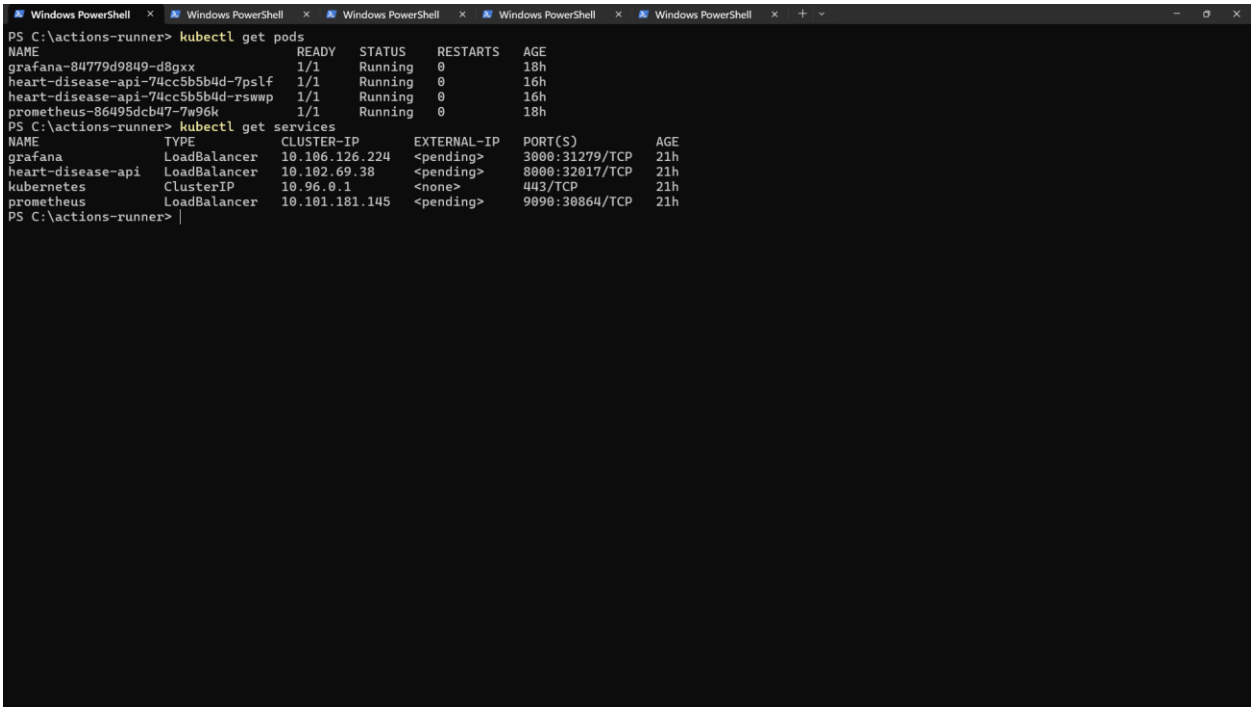

Figure 9.1 — kubectl get pods / get services showing all four workloads (API × 2 replicas, Prometheus, Grafana) running and their exposed ports.

Endpoints were verified directly via the FastAPI Swagger UI (see Figure 8.1) and via Prometheus's target health page (see Section 10), confirming the deployed service is reachable and correctly routed.

10. Monitoring & Logging

The FastAPI application exposes custom Prometheus metrics (request counts, request latency histograms, prediction counts, and a model_loaded gauge) in addition to default process metrics, scraped by a Prometheus Deployment running inside the same cluster.

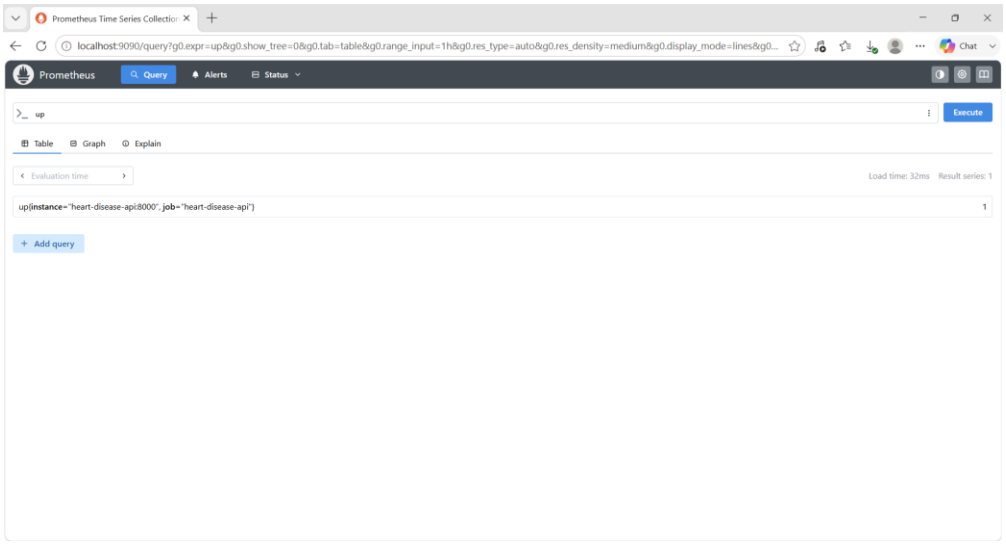

Figure 10.1 — Prometheus up query confirming the heart-disease-api scrape target is healthy (value = 1).

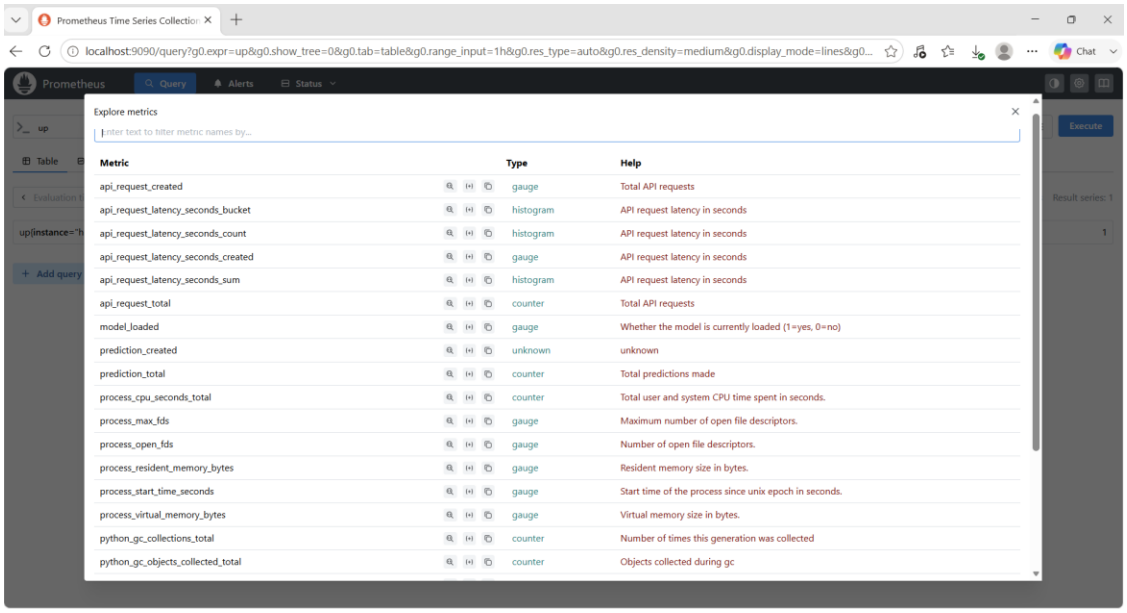

Figure 10.2 — Full list of custom and default metrics exposed by the API, discoverable via Prometheus's metric explorer.

10.1 Grafana Dashboard

A dedicated Grafana dashboard ("Heart Disease Prediction API", tagged heart-disease / mlops / prediction) visualizes request rate by endpoint, p95 latency by endpoint, 5xx error rate, prediction distribution, model load status, and cumulative request/prediction counts, refreshed every 10 seconds.

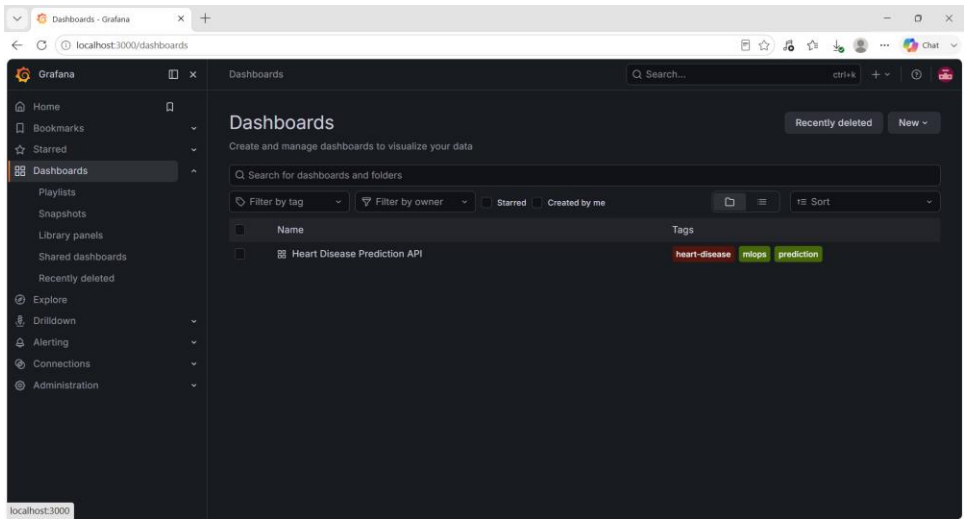

Figure 10.3 — Grafana dashboard listing.

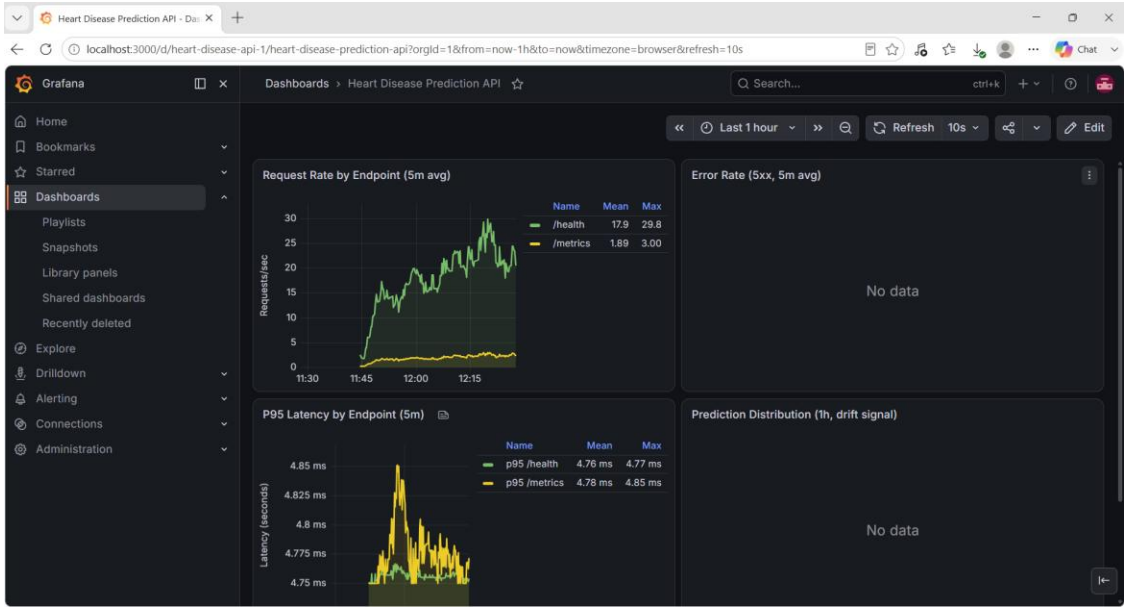

Figure 10.4 — Dashboard (last 1 hour): request rate and p95 latency by endpoint, live traffic from local testing.

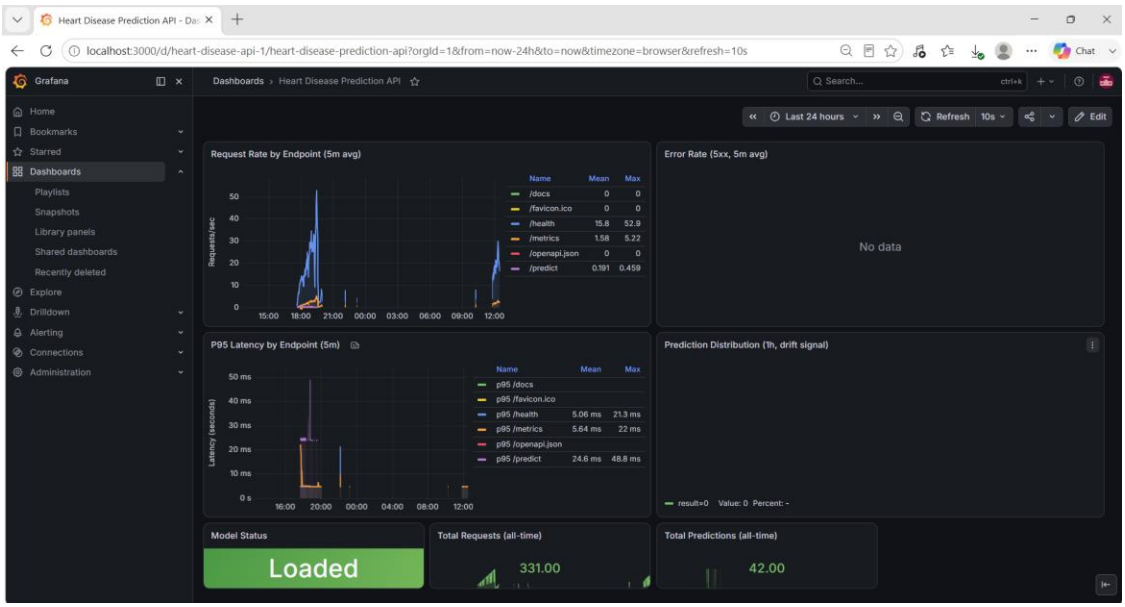

Figure 10.5 — Dashboard (last 24 hours): full endpoint breakdown including /predict, plus Model Status (Loaded), Total Requests (331), and Total Predictions (42) since deployment.

Structured JSON logging of every API request (method, path, status code, duration) is emitted by the application and visible in container logs (kubectl logs), providing a complete audit trail alongside the Prometheus/Grafana metrics stack.

11. System Architecture

The diagram below summarizes the full pipeline: from data acquisition and experiment tracking, through GitHub-hosted CI, to CD on a self-hosted runner that deploys directly into the local Kubernetes cluster, which serves the API and is continuously monitored by Prometheus and Grafana.

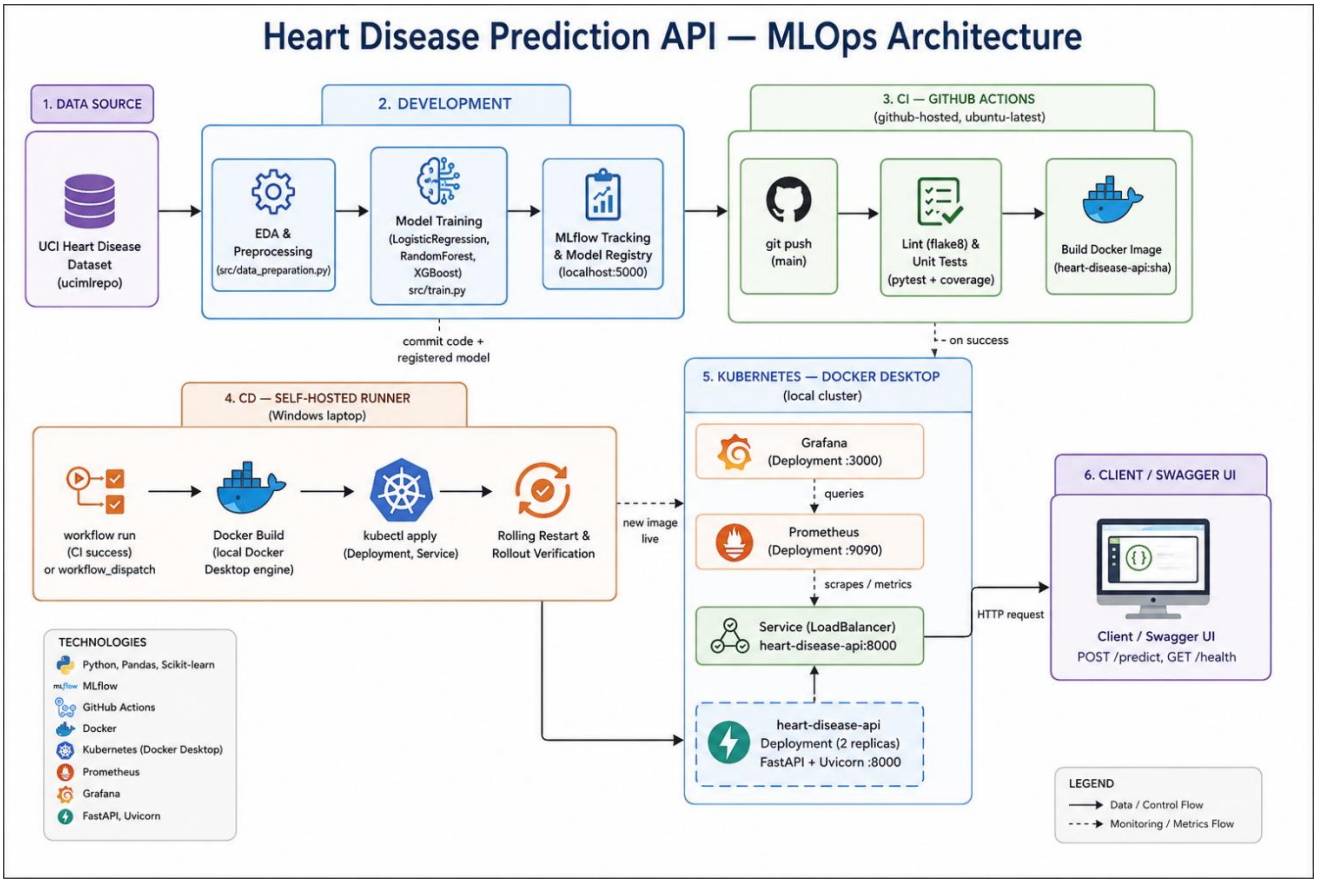

Figure 11.1 — End-to-end MLOps architecture: Development → CI (GitHub-hosted) → CD (self-hosted runner) → Kubernetes (API + Prometheus + Grafana) → Client.

Why a self-hosted CD runner?

The Kubernetes deployment target in this project is Docker Desktop's local cluster, which runs entirely on the developer's laptop and is not reachable from GitHub's cloud-hosted runners. Registering the same laptop as a self-hosted GitHub Actions runner allows the CD job to run kubectl apply and docker build using the exact same Docker engine and kubeconfig already used for manual testing, giving a genuine, working deployment step rather than a simulated one.

12. Deliverables, Video & Repository Link

12.1 Repository Structure

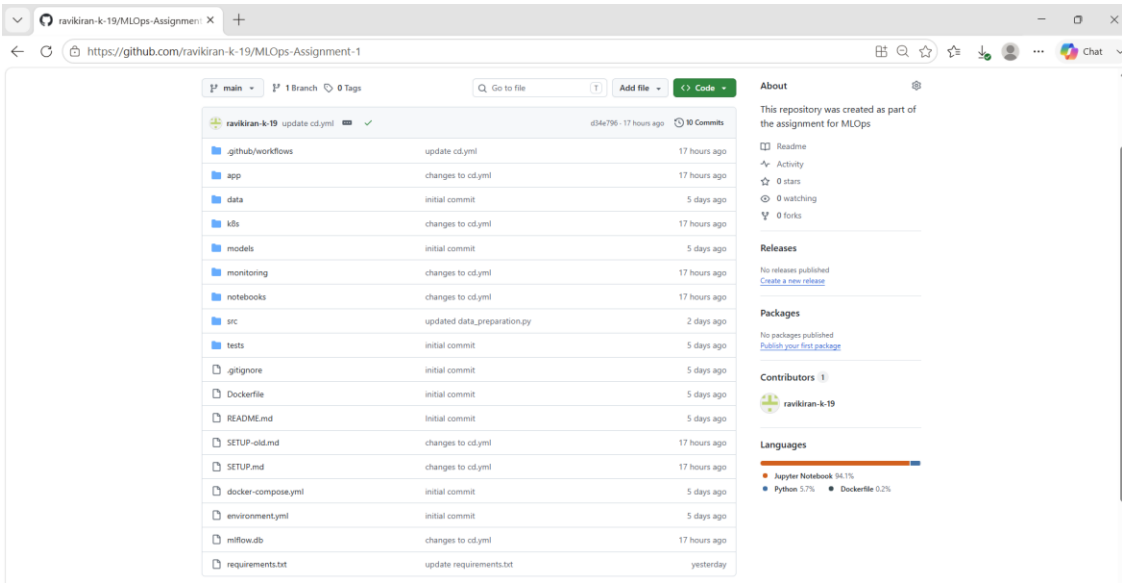

Figure 12.1 — GitHub repository structure showing all required deliverable components.

- Code, Dockerfile(s), requirements.txt / environment.yml
- Cleaned dataset access via src/data_preparation.py (UCI repo download) plus local data/ fallback
- Jupyter notebooks (notebooks/01_eda.ipynb) and training/inference scripts (src/)
- tests/ folder with unit tests, run automatically in CI
- GitHub Actions workflow YAMLs: .github/workflows/ci.yml and cd.yml
- Deployment manifests: k8s/deployment.yaml, service.yaml, prometheus-deployment.yaml, grafana-deployment.yaml
- This written report, and a screenshot folder for reporting

12.2 Repository Link

https://github.com/ravikiran-k-19/MLOps-Assignment-1

12.3 Access Instructions (Local Testing)

The API is deployed to a local Kubernetes cluster (Docker Desktop) rather than a public cloud endpoint. Access is via kubectl port-forward as documented in Section 2.4; once forwarded, the interactive API documentation is available at http://localhost:8080/docs.

12.4 Video Recording

A screen recording (MLOps-Assignment-Video.mp4) is included with this submission, demonstrating the complete pipeline end-to-end in a single live run: exploratory data analysis, model training with MLflow

experiment tracking, a code commit triggering the CI pipeline (lint, test, Docker build), the automatic hand-off to the CD pipeline on the self-hosted runner (Docker build and kubectl deployment), and finally the live, monitored API observed through Prometheus and the Grafana dashboard.

Link to the video and other documents

https://drive.google.com/drive/folders/1RCvgYrZioP3OTkY-mmqs4R2GEIOW7dfE?usp=drive_link

13. Conclusion

This project implements a complete MLOps lifecycle for a heart disease risk classifier: reproducible data preparation and EDA, comparative model development across three algorithms with full MLflow experiment tracking and model registry versioning, a Dockerized FastAPI serving layer, an automated two-stage CI/CD pipeline (GitHub-hosted CI, self-hosted CD), Kubernetes-based deployment with rolling updates, and continuous monitoring via Prometheus and Grafana. Debugging the pipeline end-to-end — particularly the self-hosted CD runner's platform-specific issues (PowerShell vs. bash shell semantics, Windows service account Docker permissions, and Service port mismatches) — provided practical, first-hand experience with the operational realities of production ML systems that go beyond model training alone.

Overall, the pipeline satisfies the assignment's production-readiness requirements: all scripts execute from a clean setup using only requirements.txt, the model serves correctly inside an isolated Docker container, and both CI and CD fail fast with clear logs on any code, test, or deployment error.